

PROJET: Explorer des heuristiques de sélection
de sous-graphe maximisant le diversité des
scénarios

PRB: on ramène uniquement le pourcentage "approx"
du sous-graphe pl. au pire chemin.

↳ ex: coût à 1000, 2 schémas à 998 et 995
vont partager le même graphe avec une petite
variation finale et donc on va refaire le même
calcul 2 fois sans apporter beaucoup d'informations.

↳ A l'inverse, un pb. de coût à 950 va être ignoré
car il est sous le seuil approx.

→ Si deux valeurs n'ont aucune info en commun
ils sont orthogonaux

SOL: Orthogonalité + clustering...
↳ nouvelle idée: je prends les N plus mauvais
BTO: je prends les N plus mauvais différents.

REVIEW: chercher l'orthogonalité, c'est chercher tous les axes
d'attaque différents.

APP: Why the arclength integral is a lower semi continuous problem?

① Chercher la distance entre 2 solutions (chemins)

①.1 Distance euclidienne:

$$d_e(x, y) = \sqrt{\sum_{i=1}^n |x_i - y_i|^2}, \forall n \in \mathbb{N}^*$$

~~①.2~~ Distance de Manhattan: distance de taxi.

$$d_n(x, y) = \sum_{i=1}^n |y_i - x_i|$$

problème car valable uniquement
sur les graphes rectangulaires.

①.3 Distance de "graphe" (type arc overlap)

idée: compter le nbr d'arc en commun
et différencier par rapport à cela.

ex: 001 et 011

dl 1 = 1 car 1 bit diff.

② Heuristique : $\left\{ \begin{array}{l} \text{borne inf} \\ \text{borne sup} \end{array} \right\}$ à regarder n $\rightarrow$ algo de Floyd
 à voir

②.1 Approche k -means :

BT: Etant donné une ensemble de points x_i , $k \in \mathbb{N}$,
le but est de partitionner les points en k classes
distinctes en **minimisant la variance**

$\hookrightarrow$ minimiser la somme des carrés
à la moyenne.

Algo de Lloyd

Def: Etant donné un ensemble de points $\{x_i\}$,

on cherche à partitionner les n points en k ensemble $S = \{S_1, \dots, S_k\}$
en minimisant la somme des carrés des distances
à la moyenne de chaque partition.

$$\arg \min \sum_{i=1}^k \sum_{x_j \in S_i} \|x_j - \mu_i\|^2, \mu_i = \text{barycentre de } S_i.$$

⚠️ indiqué comme efficace mais pas ~~opt~~ général.
Forcément optimal / $\approx$ hrs de calcul

⚠️ NP-difficile dans le cas général

Il existe d'approximation ~~par~~ polynôme par
recherche locale $\rightarrow$ pas euclidien.

ex: si $k=5$ on obtient 5 pires scénarios qui attaquent
différemment le pire de tous.

$\hookrightarrow$ pb comment choisir ce k ?

2.2. Approche Greedy:

- partir de la pire solution S_1
- chercher le pire scénario S_2 le plus éloigné de S_1
- chercher S_3 qui maximise la distance (S_1, S_2)
- jusqu'à k

↳ Problème persistant: ~~too~~ récursif ou $k \times$

↳ on aura $k+1$ solution de n à $k \times n$

↳ $k \times n$ →

→ pb de calcul.
car graph trop grand

2.3. Recy dynamique:

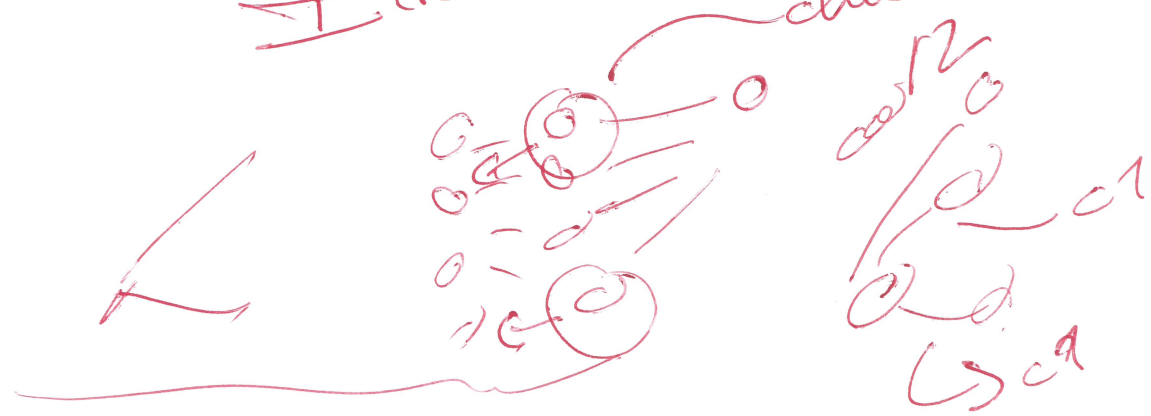
Idee: modifier du backtrack.

si on arrive à une égalité entre 2 cas P_1 de la backtrack ⇒ on prend celui qui a été le moins emprunté

↳ si 2 cas procèdent à la même chose que faire!

↳ Idee: tirage uniforme (rapide et économique)
regarder en $k-1$ mais pas forcément pas.

↳ Idee: clusterisation → la-d'encore.



Algo greedy:

Waarvoor

if feasible procedure
or complete

2. haalbaar

OK

while (selected-path.size() < N_{path} && ! candidates.empty()) {

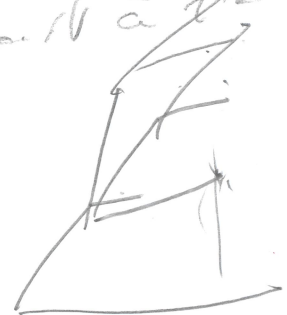
for best_max_min_dist = -1.0
for best_candidate_index = -1;) init

for (size_t c = 0; c < candidates.size(); c++) {

for (size_t s = 0; s < selected-path.size(); s++) {

for dist = calc_dist(selected-path[s], candidates[c]);
if (dist < min) {
min = dist;

probleem is dat de min
cost is de origineel.



if (min > best_max)

best_max = min;
best_candidate_index = c;

selected-path.push_back(candidates[best_candidate_index]);
candidates.erase(candidates.begin() + best_candidate_index);

Algo greedy

↳ check | max-sum disjoint
| max-min disjoint

$$\begin{aligned} \text{Si } D_H(A, B) &= |A \Delta B| \\ &= |A \cup B| - |A \cap B| \end{aligned}$$

on s'en prend :

$$\begin{aligned} D_J(A, B) &= 1 - J(A, B) = 1 - \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cup B| - |A \cap B|}{|A \cup B|} \\ &= \frac{D_H(A, B)}{|A \cup B|} \end{aligned}$$

car $J(A, B)$ est l'indice de Saccard
qui mesure la similarité

↳ $D_J(\cdot, \cdot)$ mesure lui la dissimilitude
entre 2 exemples.

Algo :

- ① Soit E l'ensemble des pires scénarios actuels
- ② Pour chaque nouveau candidat S (qui dépasse le seul décret) :
 - 2.1 Calculer D_J avec tous les $S \in E$
 - 2.2 Retenir la plus petite distance / se $S \in E$ qui lui ressemble le plus.
- ③ Sélectionner le candidat S qui maximise cette plus petite distance

P.B à prendre en compte: la hauteur.



on peut utiliser le même principe que Jaccard mais sous un autre angle:

• les $\vec{v}$ ont la même dimension T .

• $\Rightarrow$ on ne va plus regarder par où on passe mais on va regarder l'intensité de l'attaque à chaque t. $\{T\}$

Δ pas de normalisation par Jaccard

traversez autres.

OBDA-A20E

Indice de Bray-Curtis ?

utilisé en biologie pour calculer la similitude de 2 relevés:

$$S_{oc}(A, B) = \frac{2 \sum_{t=1}^T \min(\sigma_t^A, \sigma_t^B)}{\sum_{t=1}^T \sigma_t^A + \sum_{t=1}^T \sigma_t^B} \rightarrow \text{abondance niche partagée}$$

$$D_{oc}(A, B) = 1 - S_{oc} = \frac{\sum \sigma_t^A + \sum \sigma_t^B - 2 \sum \min(\sigma_t^A, \sigma_t^B)}{\sum \sigma_t^A + \sum \sigma_t^B}$$

$$= \frac{\sum |\sigma_t^A - \sigma_t^B|}{\sum \sigma_t^A + \sum \sigma_t^B} \rightarrow \text{voir preuve } \forall A, B \text{ points, c'est le cas ici}$$

$\sum \sigma_t^A \rightarrow$ budget total consacré par A $\rightarrow$ censuré.

$$\Delta_{BH} \in [0, 1]$$

0 : même composition spécifique
1 : $\emptyset$ price commune.

$\hookrightarrow$ maximiser Δ_{BH} .

$\rightarrow$ back server

~~Δ_{BH}~~

$$\Delta_{BH} = 1 - \Sigma_{BH}$$

$$= \frac{2 \sum_{t=1}^T \min(\sigma_t^A, \sigma_t^B)}{\sum_{t=1}^T \sigma_t^A + \sum_{t=1}^T \sigma_t^B}$$

$$= \frac{\sum_{t=1}^T |\sigma_t^A - \sigma_t^B|}{\sum_{t=1}^T \sigma_t^A + \sum_{t=1}^T \sigma_t^B}$$

check on

$\hookrightarrow$ ~~to~~ normalise
break

$$2.) \Delta_{BH} = \frac{\Sigma}{2\Gamma}$$

Budget d'inégalité
max

$\rightarrow$ borne sup

P_1 c'est $\neq$?